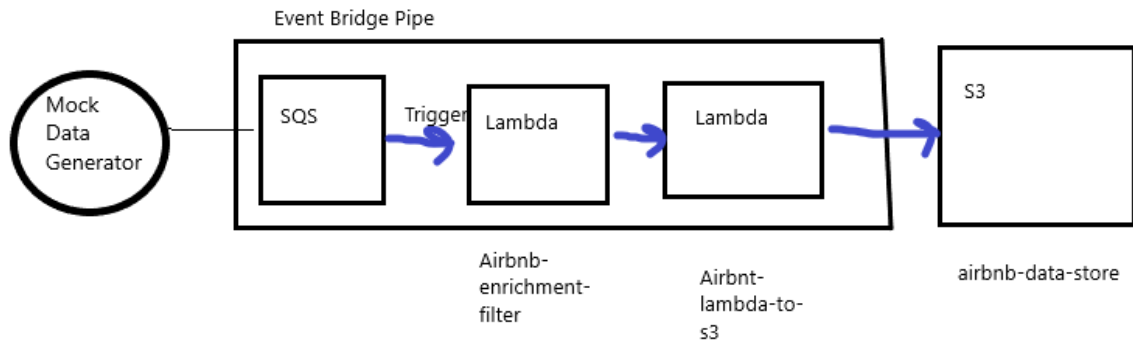


Assignment 4: Airbnb Data Stream Ingestion

Architecture Diagram:



SQS service:

Created 'AirbnbBookingQueue' with 'AirbnbBookingDLQ' as Dead letter queue.

Amazon SQS > Queues > AirbnbBookingQueue

AirbnbBookingQueue

Edit Delete Purge Send and receive messages Start DLQ redrive

Details Info

Name	Type	ARN
AirbnbBookingQueue	Standard	arn:aws:sqs:us-east-1:533267408419:AirbnbBookingQueue
Encryption	URL	Dead-letter queue
Amazon SQS key (SSE-SQS)	https://sqs.us-east-1.amazonaws.com/533267408419/AirbnbBookingQueue	Enabled

► More

Choose queue
arn:aws:sqs:us-east-1:533267408419:AirbnbBookingDLQ

Maximum receives
3
Should be between 1 and 1000

Tags - Optional [Info](#)
A tag is a label assigned to an AWS resource. Use tags to search and filter your resources or track your AWS costs.

No tags associated with this queue.

Add new tag

You can add 50 more tags.

Cancel Save

Dead-letter queue [Info](#)
Send undeliverable messages to a dead-letter queue. Also known as a redrive policy.

Edit

Queue	Maximum receives
arn:aws:sqs:us-east-1:533267408419:AirbnbBookingDLQ	3

Mock Data Generator:

1. Created a data generator which creates dictionary and sends its to queue.
2. In the Lambda function add SQS in the IAM role.
3. Use the data generator to send data to the queue.

Code:

```
import boto3
import json
import random
import string

def data_generator():
    message = {
        "bookingId":str(random.randint(10000,99999)) ,
        "userId":"'".join(random.choices(string.ascii_uppercase +
string.digits, k = 10)),
        "propertyId":"'".join(random.choices(string.ascii_uppercase +
string.digits, k = 10)),
        "location":random.choice(["Tampa, Florida","Hyd, Ind","BLR,
Ind"]),
```

```

"startDate":random.choice(["2024-03-12","2024-03-13","2024-03-14"]),
    "endDate":random.choice(["2024-03-13","2024-03-14","2024-03-15"]),
    "price":'$ ' + str(random.randint(100,999))
}
return message

def lambda_handler(event,context):
    sqs_client = boto3.client('sqs')
    for i in range(5):
        sqs_client.send_message(QueueUrl =
'https://sqs.us-east-1.amazonaws.com/533267408419/AirbnbBookingQueue',
                               MessageBody = json.dumps(data_generator()))

lambda_handler(1,2)

```

Event Bridge Pipe:

1. Created a event bridge pipe.
2. Source: SQS.

The screenshot displays the AWS EventBridge Pipes console. At the top, a visual pipeline diagram shows four stages: **Source** (SQS queue, Required), **Filtering** (Optional, Not configured), **Enrichment** (Optional, Not configured), and **Target** (Required, Not configured). Below this, the **Source** configuration page is shown with a status of "Configuration is valid".

Source Info Configuration is valid

Details
EventBridge Pipes can receive events from a variety of sources.

Source
Select a source for pipe.
SQS

SQS queue
Choose or enter the ARN of an SQS queue.
AirbnbBookingQueue

Additional settings - optional

Batch size - optional
The maximum number of messages to retrieve in a single batch.
1

Batch window - optional
The maximum amount of time to gather records before invoking the function, in seconds.
[Empty field]

- 3.
4. Removed filter and in enrichment added lambda function on enrichment filter.

5. Enrichment: Its function is to read each message from source and if “startDate == endDate” then stay is less than a day, so no need of storing it. Else return the message to the target to store those value in the table.

The screenshot shows the AWS Lambda console for the function 'AirBnB-enrichment-filter'. The breadcrumb navigation is 'Lambda > Functions > AirBnB-enrichment-filter'. The function name is displayed at the top. On the right, there are buttons for 'Throttle', 'Copy ARN', and 'Actions'. Below the function name, there are tabs for 'Function overview' (selected) and 'Info'. In the 'Function overview' section, there are buttons for 'Export to Application Composer' and 'Download'. On the left, there are tabs for 'Diagram' (selected) and 'Template'. The 'Diagram' tab shows a visual representation of the function with a box labeled 'AirBnB-enrichment-filter' and a 'Layers' section showing '(0)'. There are buttons for '+ Add trigger' and '+ Add destination'. On the right, the 'Description' section is empty. The 'Last modified' section shows '21 hours ago'. The 'Function ARN' section shows 'arn:aws:lambda:us-east-1:533267408419:function:AirBnB-enrichment-filter'. The 'Function URL' section shows a link to the function's URL.

6.

The screenshot shows the AWS Lambda console code editor for the function 'AirBnB-enrichment-filter'. The breadcrumb navigation is 'Lambda > Functions > AirBnB-enrichment-filter'. The function name is displayed at the top. On the right, there are buttons for 'Execution results' and 'Environment Variables'. The code editor shows the following Python code:

```
1 import json
2
3 def lambda_handler(event, context):
4     try:
5         print('Event: ', event)
6         print('context: ', context)
7         print(event)
8         message = json.loads(event[0]['body'])
9         print(message)
10        if (message['startDate'] == message['endDate']):
11            message = {}
12        return {
13            'message': message
14        }
15    except Exception as e:
16        return {
17            'Error message': str(e)
18        }
19
```

7.

8. Target: Target is a lambda function, which takes filtered messages and then uses lambda function to store those values in s3.
9. If file is not present then , it will create the file and store those values, else it will open the existing file and then add the data in a separate row on it.

Lambda > Functions > Airbnb-lambda-to-s3

Airbnb-lambda-to-s3

Throttle Copy ARN Actions

Function overview Info

Export to Application Composer Download

Diagram Template

Airbnb-lambda-to-s3

Layers (3)

+ Add trigger + Add destination

Description

Last modified
21 hours ago

Function ARN
arn:aws:lambda:us-east-1:533267408419:function:Airbnb-lambda-to-s3

Function URL Info

10.

```

3 import os
4 from dotenv import load_dotenv
5 from datetime import date
6 import io
7 import pandas as pd
8
9 load_dotenv()
10
11 def lambda_handler(event, context):
12     # print('Event: ', event)
13     # print('Context: ', context)
14     message = event[0]['message']
15     s3_client = boto3.client('s3')
16     s3_resource = boto3.resource('s3')
17     today_date = str(date.today())
18     print(message)
19     if (message == {}):
20         return {}
21     try:
22         print('Entering Try Fn')
23         obj = s3_client.get_object(Bucket = 'airbnb-data-store', Key = f'date={today_date}/Airbnb_{today_date}.csv')
24         obj = obj['Body'].read()
25         s=str( obj, 'utf-8')
26         data = io.StringIO(s)
27         df=pd.read_csv(data,index_col='bookingId')
28         df.loc[message['bookingId']] = [message['userId'],message['propertyId'],
29                                     message['location'],message['startDate'],message['endDate'],
30                                     message['price']]
31         df.to_csv('/tmp/test.csv',encoding='utf-8')
32         s3_resource.Bucket('airbnb-data-store').upload_file('/tmp/test.csv',f'date={today_date}/Airbnb_{today_date}.csv')
33         print(df)
34     except Exception as e:
35         print('Entering Exception')
36         print(str(e))
37         df = pd.DataFrame(columns = ['bookingId','userId','propertyId','location','startDate','endDate','price'])
38
39         df = df.set_index(list(df.columns)[0])
40         df.loc[message['bookingId']] = [message['userId'],message['propertyId'],
41                                     message['location'],message['startDate'],message['endDate'],
42                                     message['price']]
43         df.to_csv('/tmp/test.csv',encoding='utf-8')
44         s3_resource.Bucket('airbnb-data-store').upload_file('/tmp/test.csv',f'date={today_date}/Airbnb_{today_date}.csv')
45

```

11.

Output:

Example output:

bookingId	userId	propertyId	location	startDate	endDate	price
91309	88EBABTS	FRFGM44C	BLR, Ind	3/14/2024	3/13/2024	\$910
91309	88EBABTS	FRFGM44C	BLR, Ind	3/14/2024	3/13/2024	\$910
91309	88EBABTS	FRFGM44C	BLR, Ind	3/14/2024	3/13/2024	\$910
43707	IBV8U58B	DU3N7Z2X	Tampa, Fl	3/14/2024	3/15/2024	\$150

Logs Output:

Enrichment Filter:

2024-03-18T10:57:04.354+05:30	Event: [{"messageId": "1e620973-d8fd-4f49-8ce8-4120b9f91d67", "receiptHandle": "AQEBh9oDMDF5AEy18Whf5m...
2024-03-18T10:57:04.355+05:30	context: LambdaContext([aws_request_id=0ef3ea9d-c114-4e4d-8c57-3199360a3096, log_group_name=/aws/iam...
2024-03-18T10:57:04.355+05:30	[{"messageId": "1e620973-d8fd-4f49-8ce8-4120b9f91d67", "receiptHandle": "AQEBh9oDMDF5AEy18Whf5m... Copy [{"messageId": "1e620973-d8fd-4f49-8ce8-4120b9f91d67", "receiptHandle": "AQEBh9oDMDF5AEy18Whf5m...
2024-03-18T10:57:04.355+05:30	{"bookingId": "78499", "userId": "7A3IS9ACB0", "propertyId": "ZUXZ3C1C20", "location": "Tampa, Florida", "startDate": "2024-03-12", "endDate": "2024-03-14", "price": "\$ 452"}, {"attributes": {"ApproximateReceiveCount": "2", "SentTimestamp": "1710739370041", "SenderId": "AIDAXYKJW4YR7JC2DFB36", "ApproximateFirstReceiveTimestamp": "1710739385506"}, "messageAttributes": {}, "md5OfBody": "7d956b498bc6f4246ed85c3400e2d847", "eventSource": "aws:sqs", "eventSourceARN": "arn:aws:sqs:us-east-1:533267408419:AirbnbBookingQueue", "awsRegion": "us-east-1"}] Copy
2024-03-18T10:57:04.355+05:30	END RequestId: 0ef3ea9d-c114-4e4d-8c57-3199360a3096

Target Lambda:

2024-03-18T10:57:08.159+05:30	START RequestId: 33ca481d-d4ac-4ac6-abbd-d86922ee0e76 Version: \$LATEST
2024-03-18T10:57:11.007+05:30	{"bookingId": "62279", "userId": "3Z2GP9PPZS", "propertyId": "SNINQGYIM44", "location": "Tampa, Flor... Copy {"bookingId": "62279", "userId": "3Z2GP9PPZS", "propertyId": "SNINQGYIM44", "location": "Tampa, Florida", "startDate": "2024-03-12", "endDate": "2024-03-13", "price": "\$ 437"}
2024-03-18T10:57:11.007+05:30	Entering Try Fn
2024-03-18T10:57:11.427+05:30	Entering Exception
2024-03-18T10:57:11.427+05:30	An error occurred (NoSuchKey) when calling the GetObject operation: The specified key does not exist...
2024-03-18T10:57:12.369+05:30	END RequestId: 33ca481d-d4ac-4ac6-abbd-d86922ee0e76

Source Code : <https://github.com/GantaGourish/Airbnb-filter-data>